Retrieval-Augmented Generation (RAG) for Corporate Wikis/Documentation

Authors: Yasmine Huo, Tracy Ling
Team: Insight Makers

Executive Summary

[Decisions to be impacted]

This project aims to use Retrieval-Augmented Generation (RAG) to build a reliable chatbot for corporate wikis and documentation that produces contextually grounded, precise answers without retraining the underlying language model.

By utilizing RAG, we can reduce AI hallucinations and the need to retrain LLMs with new data. This would improve development and computational costs on businesses hoping to implement and run LLMs of their own, as well as helping to guarantee increased precision and credibility in their chatbot design because users can personally verify cited sources on their own. This creates better transparency and maintains trustworthiness between a company and its customers.

The decisions impacted based on this technique cover a wide range of categories, such as: technical decision, operational decisions, governance decisions, and knowledge management decisions. Users use LLMs as a tool for a wide range of activities and based on the LLM response, decisions are made.

[Business value]

This project is important to us because AI is a huge topic in the present day, with many modes of operation around the world in various sectors of work. The reliance and use of big data is increasing. Rather than focusing solely on model performance, it addresses real-world constraints such as trust, correctness, and usability. Working on this problem allows us to explore how theoretical concepts in natural language processing and data systems translate into practical tools that people can safely rely on.

[Data assets]

We use a English Wikipedia data dump snapshot as our primary dataset. The raw data was downloaded from https://dumps.wikimedia.org/enwiki/latest/enwiki-latest-pages-articles.xml.bz2

Data and Data Pre-processing

We use a English Wikipedia data dump snapshot as our primary dataset. The raw data was downloaded from https://dumps.wikimedia.org/enwiki/latest/enwiki-latest-pages-articles.xml.bz2 and was extracted and cleaned using a Python script called WikiExtractor. For each article, the text is broken up into chunks. The chunk metadata is as shown below:

Corpus Data Description per chunk: | Variable | Description | | --- | --- | | chunk_id | A unique, deterministic identifier for the chunk | | page_id | Wikipedia's unique identifier for the article | | title | Title of the article | | section_path | A list representing the hierarchical section structure of the article | | paragraph_index | The index of the paragraph within the article | | text | Text contained within the chunk | | source | Origin of dataset | | url | URL link of the specific Wikipedia article | | revid | Article revision ID for the particular version of the article text |

There is no standard chunk size within a corpus, since each Wikipedia article varies in size. However, only paragraphs within an article with more than 1200 characters are futher split into chunks.

In [ ]:
```
import json

raw_data = []
with open('../data/processed/corpus_chunks.jsonl', 'r') as f:
    for line in f:
        raw_data.append(json.loads(line.strip()))

raw_data[0]
```

Out[ ]:
```
{'chunk_id': 'wiki_c05c4f78f1ae',
 'page_id': '709145',
 'title': 'Ariadne (software)',
 'section_path': ['(root)'],
 'paragraph_index': 0,
 'text': 'The European Knowledge Pool System Ariadne (acronym for Alliance
of Remote Instructional Authoring and Distribution Networks for Europe) is
a European association (or consortium) for sharing knowledge and fostering
international cooperation in teaching that is open to the world.',
 'source': 'enwiki_pages_articles_latest',
 'url': 'https://en.wikipedia.org/wiki?curid=709145',
 'revid': '15996738'}
```

In [2]:
```
print("Number of chunks:", len(raw_data))
```

```
Number of chunks: 591
```

Since the dataset contains an extremely diverse variety of article topics, we decided to reduce our dataset to only Computer Science subjects and specialize our initial chatbot model to only answering queries related to this subject. To filter the data, we first use an

elementary keyword filter to roughly reduce our initial dataset size to generally relevant texts.

In [3]:
```
# CS Keywords for filtering
cs_keywords = [
    'computer', 'programming', 'algorithm', 'software', 'data structure', 'm
    'artificial intelligence', 'ai', 'database', 'network', 'operating syste
    'programming language', 'code', 'computing', 'informatics', 'cybersecuri
    'cryptography', 'robotics', 'virtual reality', 'augmented reality', 'coc
    'big data', 'quantum computing', 'java', 'python', 'c++', 'javascript',
    'ram', 'cpu', 'gpu', 'software engineering'
]

filtered_data = [
    item for item in raw_data
    if any(keyword in item['title'].lower() or keyword in item['text'].lower
]

print(f"Filtered data size: {len(filtered_data)}")
```

```
Filtered data size: 354
```

Next, we use the TF-IDF (Term Frequency-Inverse Document Frequency) Vectorizer which is a statistical measure used to evaluate the importance of a word in one document in relation to a collection of documents. This is calculated by both how often a word appears in a document and by reducing the weights of common words across multiple documents.

To evaluate how well the vectorizer did, we used the cosine similarity between each document (threshold = 0.1) to determine how similar a document is to the CS-related terms. The score is between 0 and 1, with scores closer to 1 indicating high similarities and scores closer to 0 with low similarities.

In [23]:
```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

# Use only title and text for TF-IDF analysis
title_texts = [f"{item['title']} {item['text']}" for item in filtered_data]
# print(title_texts[0])
# print(max(len(text) for text in title_texts))
# print(f"Combined texts for TF-IDF: {len(title_texts)} items")

cs_reference = " ".join(cs_keywords)
# print(cs_reference)

vectorizer = TfidfVectorizer(max_features=700, stop_words='english', lowerca
all_docs = title_texts + [cs_reference]
tfidf_matrix = vectorizer.fit_transform(all_docs)

# Cosine similarity between each document and the CS reference
# Values close to 1 indicate high similarity, while values close to 0 indica
# High score indicates the document is more relevant to CS topics, while a l
```

```
cs_ref_vector = tfidf_matrix[-1]
similarities = cosine_similarity(tfidf_matrix[:-1], cs_ref_vector).flatten()

# Filter documents based on similarity threshold
threshold = 0.1
high_scores = np.where(similarities >= threshold)[0]
tfidf_filtered_data = [filtered_data[i] for i in high_scores]

print(f"After TF-IDF filtering (threshold={threshold}): {len(tfidf_filtered_
print(f"Similarity scores - Min: {similarities.min():.3f}, Max: {similaritie
```

```
After TF-IDF filtering (threshold=0.1): 18 items
Similarity scores - Min: 0.000, Max: 0.229, Mean: 0.026
```

Model Updates

At this stage, we have moved from basic preprocessing toward a more complete agentic RAG pipeline.

- **Baseline filtering:** We used keyword filtering and TF-IDF style relevance scoring to narrow the corpus toward computer science–related content.
- **Retriever:** Our next retrieval backbone is dense embedding-based retrieval for semantic search.
- **Reranker:** We plan to use a Cross-Encoder reranker to improve top-k precision.
- **Generator: Qwen-2.5** will be used for grounded answer generation based on retrieved evidence.
- **Workflow: LangGraph** will coordinate retrieval, evaluation, and generation in a structured pipeline.
- **Reliability check:** We also plan to add an evaluator to decide whether the system should answer, refine retrieval, or abstain.

Machine Learning Morphism (MLM)

To conclude our project breakdown, we present our machine learning workflow in the form of a Machine Learning Morphism (MLM): Input data is morphed into reliable outputs that can be used to augment decision-making processes.

Firstly, our workflow ingests data through its **input layer**. In the project at hand, this input layer is represented by an English Wikipedia data dump, which serves as proxy data for mass-scale corporate documents. Since the articles scraped from Wikipedia contain noisy and overly general information, we route these documents through a **data preprocessing layer** that cleans and filters them towards computer science–related articles. During preprocessing, documents are chunked into smaller passages with additional metadata attributes attached to each chunk such as title, section path, and source wiki link.

Processed passages then flow into the **representation layer** where passages are transformed into their dense vector embeddings. With each chunk now represented as a vector, we can numerically describe semantic meaning and relate user queries to chunks of documents via vector space operations.

Following representation, processed chunks head into the **retrieval layer**. Upon receiving a user query, the retriever fetches the most semantically similar chunks from the vector database. To increase result precision, retrieved candidates are sent through a **reranking layer**. Here, a cross-encoder reranker rearranges the order of retrieved passages and elevates the most relevant evidence to the top of the list.

Top-k passages are then sent to the **generation layer** where the generator synthesizes a response grounded on retrieved evidence. Rather than freely answering from the model's own memory, our generator can articulate precise answers that reference external context.

Finally, the workflow undergoes an **evaluation and control layer**. For this project, an evaluator or agentic controller determines if enough evidence has been provided to validate an answer. If not, the agent may edit its retrieval, abstain from answering the question, or ask for a better chain of reasoning. This final step is critical as it improves the reliability, mitigates hallucination, and boosts the trustworthiness of the generated response.

We can therefore summarize this Machine Learning Morphism as such:

Raw documents → preprocessing and chunking → embedding representation → retrieval → reranking → grounded generation → evaluation/control → reliable answer

This morphism allows us to visualize how unstructured text is transformed into transparent and decision-useful responses.

Next Steps

Our immediate next step after preprocessing is to build the dense retrieval module.

Immediate Next Step: Dense Retrieval

- Encode each filtered chunk into a dense embedding.
- Store chunk embeddings in a vector index.
- Retrieve top-k semantically relevant chunks for each query.
- Evaluate retrieval quality under different top-k settings.

This is the most important next step because our current filtering pipeline is still largely lexical. Dense retrieval will allow the system to match queries and documents based on

semantic meaning, which is more suitable for documentation-style QA.

Later Steps

- Add a Cross-Encoder reranker to improve retrieval precision.
- Connect retrieved evidence to Qwen-2.5 for grounded generation.
- Use LangGraph to organize the full workflow.
- Add an evaluator so the system can answer, retrieve again, or abstain.
- Evaluate the full pipeline using RAGAS metrics.

Timeline of Next Steps

Week 1: Finalize preprocessing pipeline to filter docs/chunks of interest and structure metadata for Wikipedia-derived corpus. Verify data quality signals. Output cleaned list of chunks for use downstream in retrieval processes.

Week 2: Develop & test dense retrieval by creating embeddings for each chunk and populating vector database/index. Measure if retriever provides relevant passages given example user queries.

Week 3: Add a reranking model to improve precision. Measure retrieved results before and after reranking to identify if useful evidence is being ranked highest.

Week 4: Hook up retrieval pipeline to generative model to enable system to produce grounded answers. Begin testing answer quality, source citation patterns, grounding of generated responses to retrieved docs.

Week 5: Add evaluator/agentic controller to assess whether retrieved evidence is sufficient for supporting an answer. Enable agent to know when to answer, when to rerank/retrieve, when to reject to minimize hallucinations.

Week 6: Run full suite of end-to-end evaluations with chosen metrics/tools (e.g., RAGAS). Diagnose/review quality of retrieval, answer correctness and grounding, overall reliability, and iterate accordingly.

Source Code

https://github.com/Washu-Project-Rag/project-rag